PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE — 4th Semester
Aug – Dec 2026

UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

INTELLIGENT DATABASE DISCOVERY AND OPTIMIZATION PLATFORM

(IDOP)

Submitted by:

SRN	Student Name
PES1UG24AM191	Piyush Shiv
PES1UG24AM215	Raghav G Uddihal

DBMS Lab Faculty: Prof. Devna Sumesh

Abstract

Modern applications generate large volumes of structured database metadata, SQL workloads, query execution information, and technical documentation. This makes it difficult for developers and database administrators to understand database systems, discover related queries, and identify potential optimization opportunities. The Intelligent Database Discovery and Optimization Platform (IDOP) combines MySQL with the Qdrant vector database to support structured database exploration and semantic SQL-workload discovery. MySQL stores application users, database sources, schema objects, query workloads, optimization recommendations, and audit records. Qdrant stores vector representations of SQL workloads and retrieves semantically similar records using cosine similarity. A Next.js web interface communicates with a FastAPI backend, while Python modules coordinate database access, workload analysis, embedding generation, and retrieval. The project demonstrates relational design, SQL operations, advanced database features, vector similarity search, application integration, and role-based access control.

Table of Contents

Sl. No	Topic	Page
1	Introduction – Problem Statement	3
2	User Requirements Specification – Tech Stack	3
3	E-R Model	4
4	Relational Model	5
5	SQL DDL Statements	6
6	SQL DML Statements	7
7	NoSQL / Vector Database Commands	8
8	Results – Resulting Tables' Screenshots	9
9	Next.js Project Tree Structure	9
10	Application Code and Demonstration	10–17
11	Evaluation Rubrics – Completion Status	18

1. Introduction

Problem Statement

Database teams work with relational schemas, SQL workloads, execution plans, and optimization history that are often distributed across separate tools. Exact-text search is insufficient when a user remembers the intent of a query but not its precise SQL syntax. IDOP addresses this problem by providing a unified interface to inspect database metadata, analyze stored SQL workloads, retrieve semantically related queries, and review optimization recommendations alongside structured records.

Objectives

Maintain structured metadata for database sources, schema objects, workloads, recommendations, and audit events; analyze SQL workload patterns and produce explainable recommendations; support natural-language workload discovery using embeddings and similarity search; present workload details and MySQL execution plans through a web interface; and enforce role-based access for Viewer, Analyst, and Admin users.

2. User Requirements Specification – Tech Stack

The platform allows authenticated users to explore schemas and workloads, search semantically related SQL statements, inspect recommendations and execution-plan information, and manage users according to their assigned roles. The backend mediates access to MySQL and Qdrant; credentials and sensitive configuration are supplied through environment variables.

Technology	Role and justification
MySQL 8.4.11	Relational source of truth for demo and platform databases, constraints, SQL operations, indexes, routines, views, transactions,
Python	Workload analysis, database access, workload processing, and semantic-search orchestration.
FastAPI / Uvicorn	Backend REST API and ASGI server between the web application and data layer.
Qdrant	Vector database storing workload embeddings and payloads for semantic retrieval.
FastEmbed	Embedding generation using sentence-transformers/all-MiniLM-L6-v2 (384-dimensional vectors).
Next.js / React / TypeScript	Frontend application, page routing, and interactive user interface.
Docker	Local Qdrant service runtime with persistent storage.
mysql-connector-python / python-dotenv	MySQL connectivity and environment configuration.

Choice of vector database

SQL workload text is unstructured relative to the platform’s relational metadata and may be described by users in different words. Qdrant stores embeddings and payload metadata in a dedicated collection, enabling semantic retrieval without replacing MySQL as the authoritative store. FastEmbed converts a natural-language query into a vector; Qdrant retrieves nearest workload vectors using cosine similarity; returned workload identifiers connect results to structured MySQL records.

3. E-R Model

List of entities

Demo database (idop_demo_db): CUSTOMER, PRODUCT, ORDER, ORDER_ITEM, PAYMENT.
Platform database (idop_platform_db): USER, DATA_SOURCE, SCHEMA_OBJECT, QUERY_WORKLOAD, OPTIMIZATION_RECOMMENDATION, AUDIT_LOG.
No weak entities are identified; each listed entity has its own primary key.

Non-simple attributes

The conceptual diagrams use simple attributes. Unique attributes are marked in the diagrams to indicate uniqueness constraints, not separate entities. SQL data types are omitted from the conceptual ER diagrams. JSON audit details are an implementation-level representation.

ER diagram snapshots

Demo database — idop_demo_db

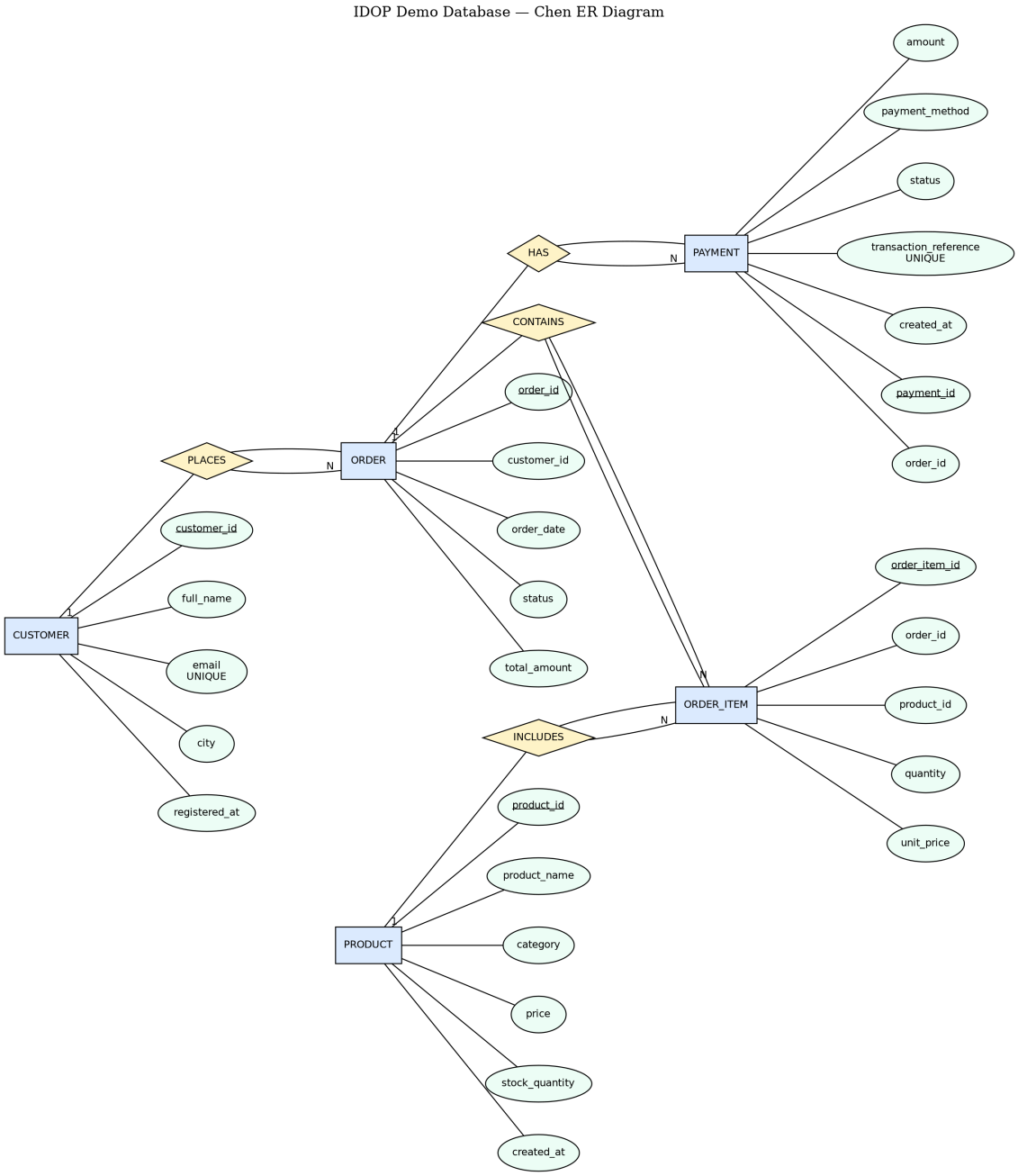


Figure 1. Chen ER diagram for the e-commerce demonstration database.

Relationships: CUSTOMER 1:N ORDER; ORDER 1:N ORDER_ITEM; PRODUCT 1:N ORDER_ITEM; ORDER 1:N PAYMENT.

Platform database — idop_platform_db

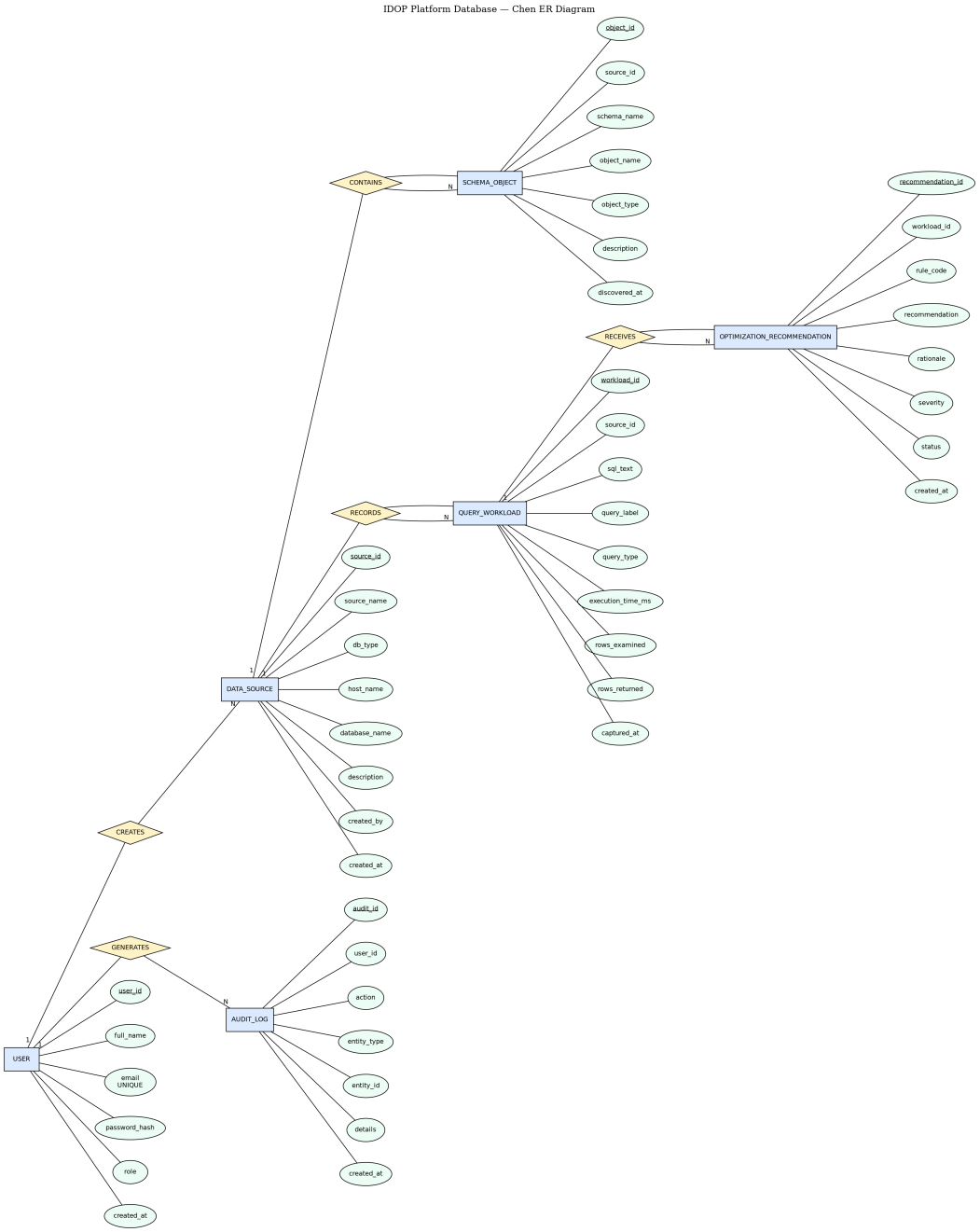


Figure 2. Chen ER diagram for the IDOP platform metadata database.

Relationships: USER 1:N DATA_SOURCE; DATA_SOURCE 1:N SCHEMA_OBJECT; DATA_SOURCE 1:N QUERY_WORKLOAD; QUERY_WORKLOAD 1:N OPTIMIZATION_RECOMMENDATION; USER 1:N AUDIT_LOG. Nullable foreign keys indicate optional participation where applicable.

4. Relational Model

List of relations and mapping

The relational model maps each strong entity to a relation. ORDER_ITEM is an associative relation between ORDER and PRODUCT and stores line-level quantity and unit price. Primary keys identify tuples; foreign keys implement the relationships shown in the ER diagrams.

Database	Relation	Primary key	Foreign key / constraints
Demo	customers	customer_id	email UNIQUE
Demo	products	product_id	—
Demo	orders	order_id	customer_id → customers.customer_id
Demo	order_items	order_item_id	order_id → orders.order_id; product_id → products.product_id
Demo	payments	payment_id	order_id → orders.order_id; transaction_reference UNIQUE
Platform	users	user_id	email UNIQUE; role constrained
Platform	data_sources	source_id	created_by → users.user_id (nullable)
Platform	schema_objects	object_id	source_id → data_sources.source_id
Platform	query_workloads	workload_id	source_id → data_sources.source_id
Platform	optimization_recommendations	recommendation_id	workload_id → query_workloads.workload_id
Platform	audit_logs	audit_id	user_id → users.user_id (nullable)

Relations not represented as standalone ER entities

ORDER_ITEM is the associative relation implementing the many-to-many conceptual association between ORDER and PRODUCT. No additional standalone relation is introduced solely for a multivalued attribute in the supplied schema.

Schema diagram

The relation table above summarizes the relational schema and identifies primary and foreign keys. The repository contains the SQL schema and ERD source/rendered files.

5. SQL DDL Statements

The implementation creates idop_demo_db for the e-commerce demonstration and idop_platform_db for platform metadata. DDL defines tables, primary keys, foreign keys, unique/check constraints, indexes, a view, stored routines, and a trigger. The following is a representative excerpt; the complete executable SQL is maintained in the GitHub repository.

Representative CREATE TABLE statements

```
CREATE TABLE customers (
  customer_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  city VARCHAR(100) NOT NULL,
  registered_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE orders (
  order_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  customer_id BIGINT UNSIGNED NOT NULL,
  order_date DATETIME NOT NULL,
  status VARCHAR(30) NOT NULL,
  total_amount DECIMAL(12,2) NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Other implemented DDL objects

The schema also defines the remaining demo and platform tables, indexes for common lookup paths, the v_customer_order_summary view, fn_order_item_total function, sp_reconcile_order_total procedure, and an AFTER UPDATE trigger on optimization_recommendations that records status changes in audit_logs.

6. SQL DML Statements

Seed data uses INSERT operations. The application and demonstration use SELECT and UPDATE operations, including joins, aggregation, nested queries, and transaction-controlled changes.

Join and aggregation

```
SELECT c.full_name,
       COUNT(o.order_id) AS total_orders,
       SUM(o.total_amount) AS total_order_value
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.full_name;
```

Nested query

```
SELECT order_id, customer_id, total_amount, status
FROM orders
WHERE total_amount >
      (SELECT AVG(total_amount) FROM orders)
ORDER BY total_amount DESC;
```

Additional SQL operations

The customer-order summary query returned 10 rows. fn_order_item_total(1) returned 4798.00; sp_reconcile_order_total(1) completed successfully. The view returned 10 customer rows. An AFTER UPDATE trigger generated a RECOMMENDATION_STATUS_CHANGED audit event. A transaction changed order 8 to CANCELLED and ROLLBACK restored it to PENDING. FULLTEXT search returned three matching workload rows. EXPLAIN selected idx_orders_customer_date and estimated three rows; this is an optimizer plan, not a measured benchmark.

7. NoSQL / Vector Database Commands

Qdrant is used for semantic discovery of SQL workloads. The collection sql_workloads uses cosine distance and stores workload metadata as point payloads. FastEmbed generates 384-dimensional vectors using sentence-transformers/all-MiniLM-L6-v2.

Collection configuration and upsert — representative Python operations

```
COLLECTION = "sql_workloads"
vectors_config = models.VectorParams(
    size=vector_size,
    distance=models.Distance.COSINE
)
client.upsert(
    collection_name=COLLECTION,
    points=[models.PointStruct(
        id=workload_id,
        vector=embedding,
        payload={
            "workload_id": workload_id,
            "query_label": query_label,
            "sql_text": sql_text
        }
    )],
    wait=True
)
```

Similarity search

```
query_vector = next(
    embedding_model.query_embed(user_query)
).tolist()
results = client.query_points(
    collection_name="sql_workloads",
    query=query_vector,
    limit=limit,
    with_payload=True
).points
```

Retrieval strategy

A natural-language query is embedded using FastEmbed. Qdrant returns nearest workload vectors and their payloads using cosine similarity. The returned workload identifiers link to structured MySQL workload and recommendation records. The captured collection response reports a healthy collection, four points, and vector size 384.

8. Results

Resulting tables and observed outputs

Area	Observed result
MySQL schemas	SHOW TABLES confirms the expected objects in both databases.
Join and aggregation	Customer-order summary query returned 10 rows.
Stored function / procedure	Function returned 4798.00; procedure completed successfully.
View	Customer summary view returned 10 rows.
Trigger / audit	Audit log contains a recommendation status-change event.
Transaction	ROLLBACK restored order 8 to PENDING.
FULLTEXT search	Three matching workload records returned.
EXPLAIN	idx_orders_customer_date selected; estimated three rows.
Qdrant	Healthy collection; cosine distance; 384 dimensions; four points.
Tests / build	10 Python tests passed; Next.js production build succeeded.

The demonstration workload set is small and seeded. EXPLAIN provides optimizer estimates, not before/after runtime measurements.

9. Next.js – Project Tree Structure

```
frontend/
|-- app/
|   |-- admin/users/page.tsx
|   |-- api/admin/users/route.ts
|   |-- dashboard/page.tsx
|   |-- queries/[id]/page.tsx
|   |-- schemas/page.tsx
|   |-- search/page.tsx
|   |-- signup/page.tsx
|   |-- workloads/page.tsx
|   |-- layout.tsx
|   `-- page.tsx
|-- components/Navbar.tsx
|-- lib/api.ts
|-- next.config.ts
|-- package.json
`-- tsconfig.json
```

Application pages and roles

Login/Signup supports authentication and Viewer registration. Dashboard summarizes workloads, databases, recommendations, and system status. Schemas displays database/table metadata. Workloads lists captured SQL statements. Semantic Search retrieves related SQL workloads. Query Details displays workload information, recommendations, and execution-plan details. User Management supports administrator account provisioning.

Authentication and authorization

The application supports Viewer, Analyst, and Admin roles. Passwords are stored as hashes and authenticated requests use signed tokens. Authorization checks are enforced by the backend. Viewer can access dashboard, schemas, and workloads; Analyst adds semantic search and recommendation analysis/status operations; Admin adds user management and administrative provisioning.

10. Sample Application Code and Demonstration Screenshots

The following figures provide application and database demonstration evidence, including all supplied screenshots.

Figure 1. Login interface — IDOP Console sign-in and account creation.

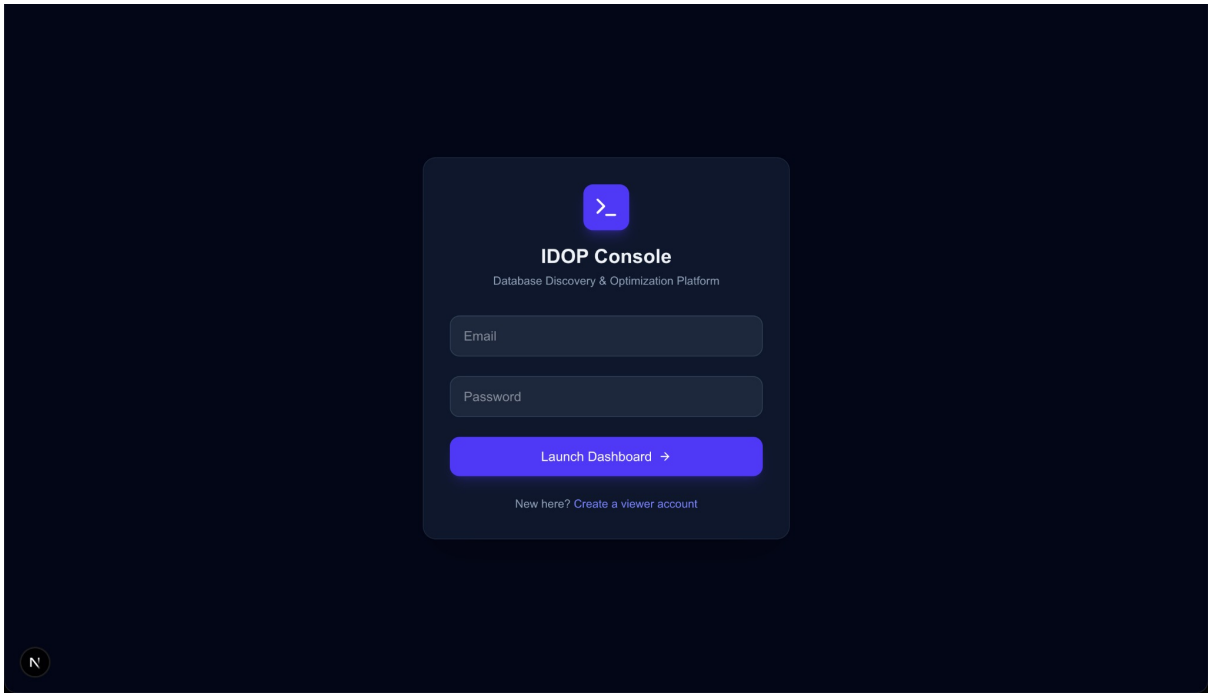


Figure 2. System dashboard — workload count, active database, pending recommendations, and system-health indicator.

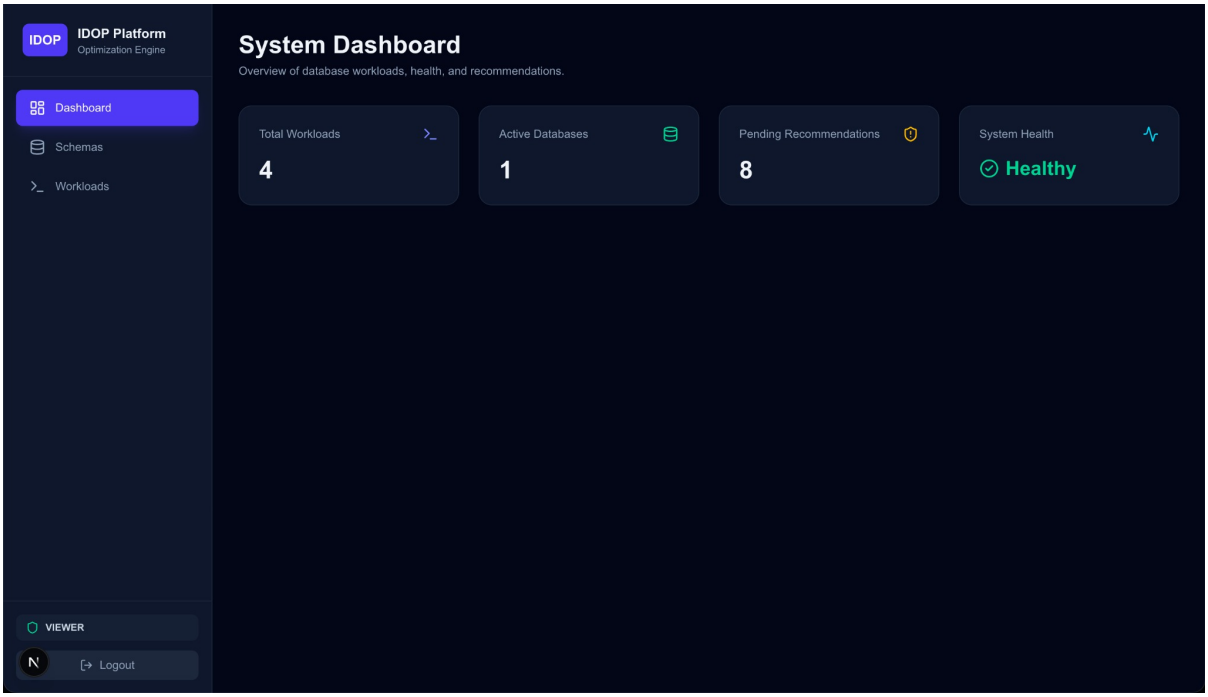


Figure 3. Database and schema explorer — demo database tables, columns, row counts, and key indicators.

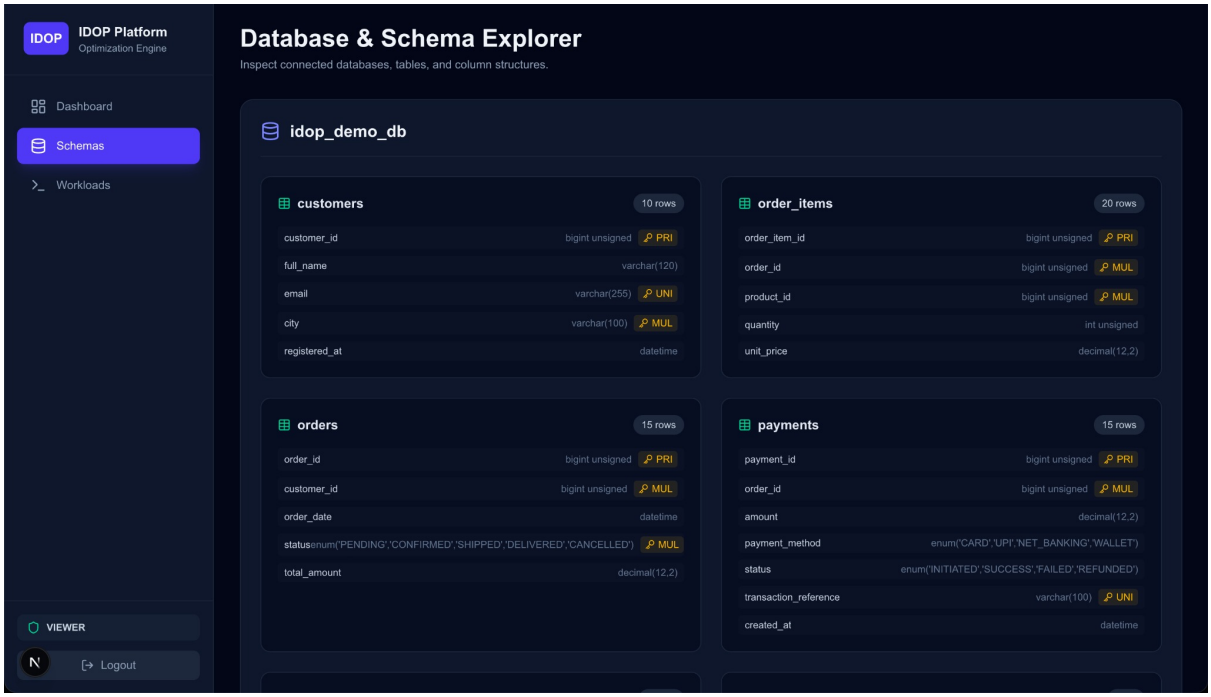


Figure 4. SQL workload explorer — captured SQL workloads and analysis actions.

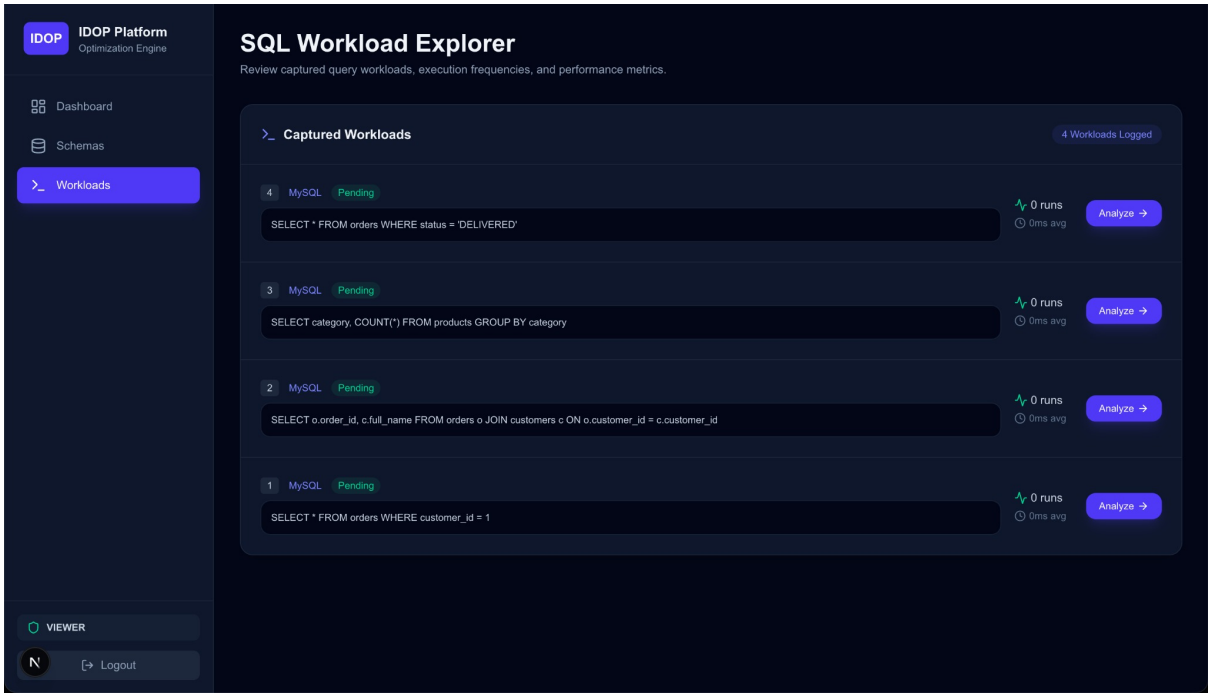


Figure 5. Semantic search — natural-language query and ranked workload matches with similarity scores.

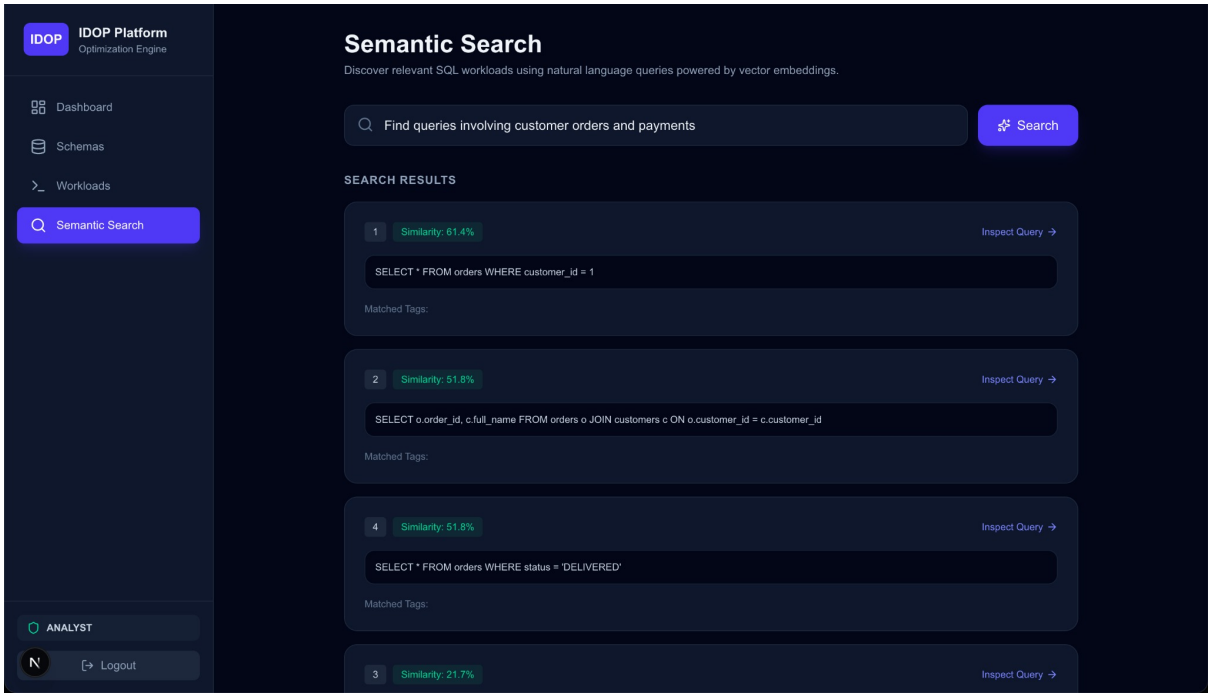


Figure 6. User management — administrator interface and role selection.

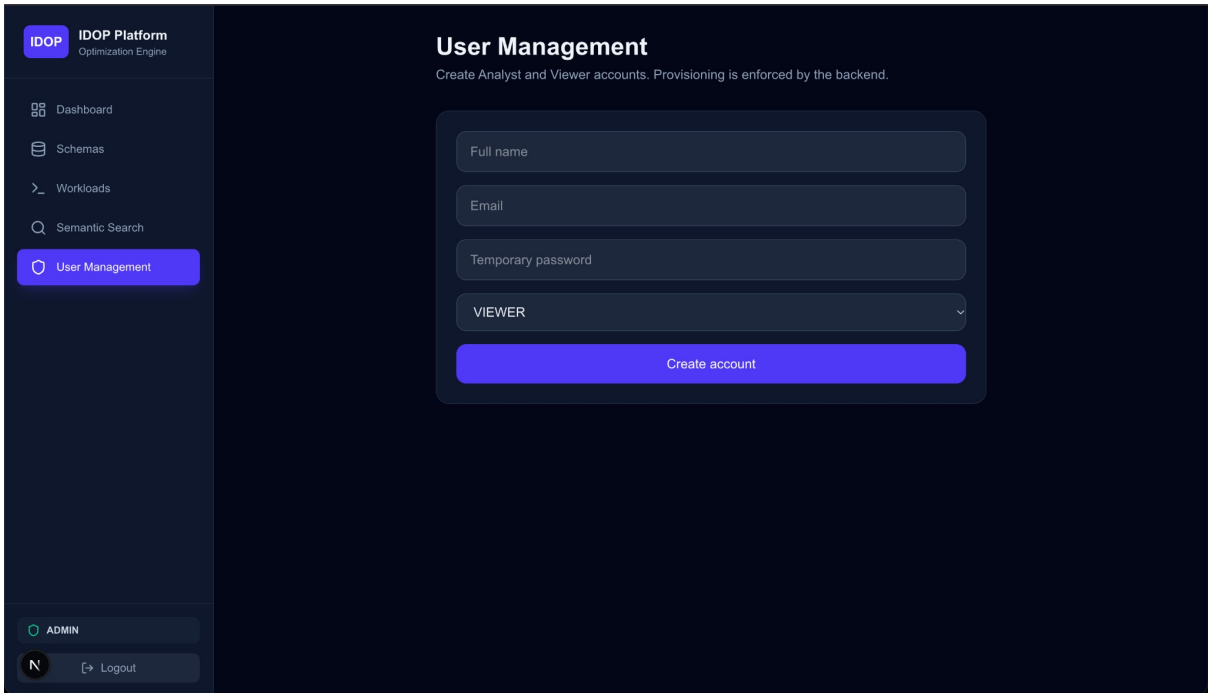
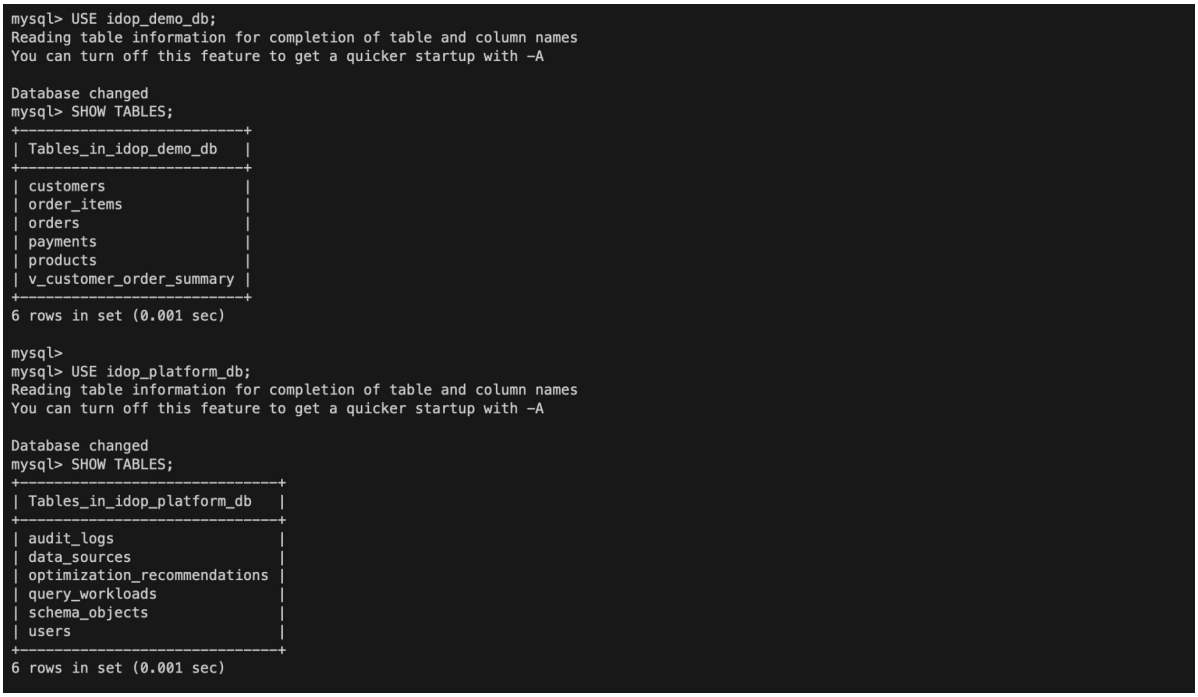


Figure 7. MySQL database objects — SHOW TABLES output for both databases.



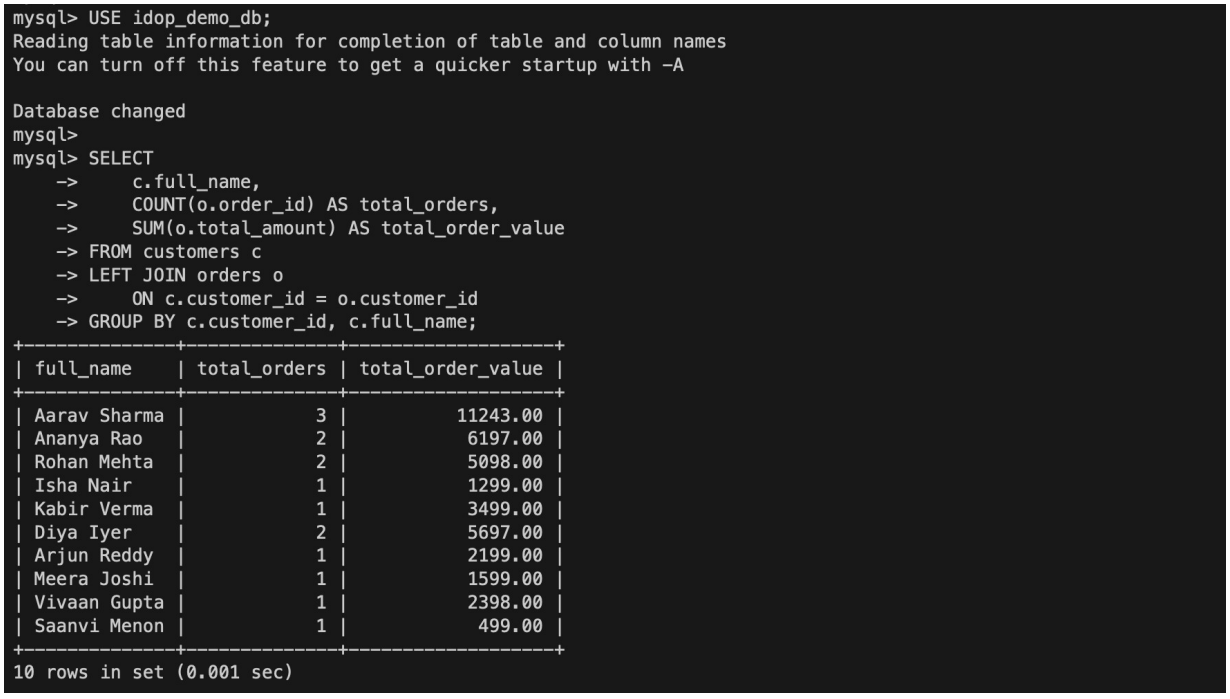
```
mysql> USE idop_demo_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+--------------------------+
| Tables_in_idop_demo_db   |
+--------------------------+
| customers                |
| order_items              |
| orders                   |
| payments                 |
| products                 |
| v_customer_order_summary |
+--------------------------+
6 rows in set (0.001 sec)

mysql>
mysql> USE idop_platform_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----------------------------+
| Tables_in_idop_platform_db  |
+-----------------------------+
| audit_logs                  |
| data_sources                |
| optimization_recommendations |
| query_workloads             |
| schema_objects              |
| users                       |
+-----------------------------+
6 rows in set (0.001 sec)
```

Figure 8. Join and aggregation — LEFT JOIN query returning customer order counts and values.



```
mysql> USE idop_demo_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
mysql> SELECT
    ->     c.full_name,
    ->     COUNT(o.order_id) AS total_orders,
    ->     SUM(o.total_amount) AS total_order_value
    -> FROM customers c
    -> LEFT JOIN orders o
    ->     ON c.customer_id = o.customer_id
    -> GROUP BY c.customer_id, c.full_name;
+--------------+--------------+--------------------+
| full_name    | total_orders | total_order_value  |
+--------------+--------------+--------------------+
| Aarav Sharma |            3 |           11243.00 |
| Ananya Rao   |            2 |            6197.00 |
| Rohan Mehta  |            2 |            5098.00 |
| Isha Nair    |            1 |            1299.00 |
| Kabir Verma  |            1 |            3499.00 |
| Diya Iyer    |            2 |            5697.00 |
| Arjun Reddy  |            1 |            2199.00 |
| Meera Joshi  |            1 |            1599.00 |
| Vivaan Gupta |            1 |            2398.00 |
| Saanvi Menon |            1 |             499.00 |
+--------------+--------------+--------------------+
10 rows in set (0.001 sec)
```

Figure 9. Stored function and procedure — function result and successful procedure execution.

```
mysql> SELECT *
    -> FROM idop_demo_db.v_customer_order_summary
    -> LIMIT 10;
+-------------+--------------+--------------------------+-------------+--------------------------+---------------------+
| customer_id | full_name    | email                    | order_count | non_cancelled_order_value | last_order_date     |
+-------------+--------------+--------------------------+-------------+--------------------------+---------------------+
|           1 | Aarav Sharma | aarav.sharma@example.com |           3 |                 11243.00 | 2025-09-08 11:20:00 |
|           2 | Ananya Rao   | ananya.rao@example.com   |           2 |                  6197.00 | 2025-08-28 09:30:00 |
|           3 | Rohan Mehta  | rohan.mehta@example.com  |           2 |                  5098.00 | 2025-09-02 12:45:00 |
|           4 | Isha Nair    | isha.nair@example.com    |           1 |                     0.00 | 2025-08-10 17:10:00 |
|           5 | Kabir Verma  | kabir.verma@example.com  |           1 |                  3499.00 | 2025-08-12 11:00:00 |
|           6 | Diya Iyer    | diya.iyer@example.com    |           2 |                  3798.00 | 2025-09-05 15:35:00 |
|           7 | Arjun Reddy  | arjun.reddy@example.com  |           1 |                  2199.00 | 2025-08-18 16:25:00 |
|           8 | Meera Joshi  | meera.joshi@example.com  |           1 |                  1599.00 | 2025-08-20 10:50:00 |
|           9 | Vivaan Gupta | vivaan.gupta@example.com |           1 |                  2398.00 | 2025-08-22 14:15:00 |
|          10 | Saanvi Menon | saanvi.menon@example.com |           1 |                   499.00 | 2025-08-25 18:00:00 |
+-------------+--------------+--------------------------+-------------+--------------------------+---------------------+
10 rows in set (0.001 sec)
```

Figure 10. Customer summary view — customer-level aggregates from v_customer_order_summary.

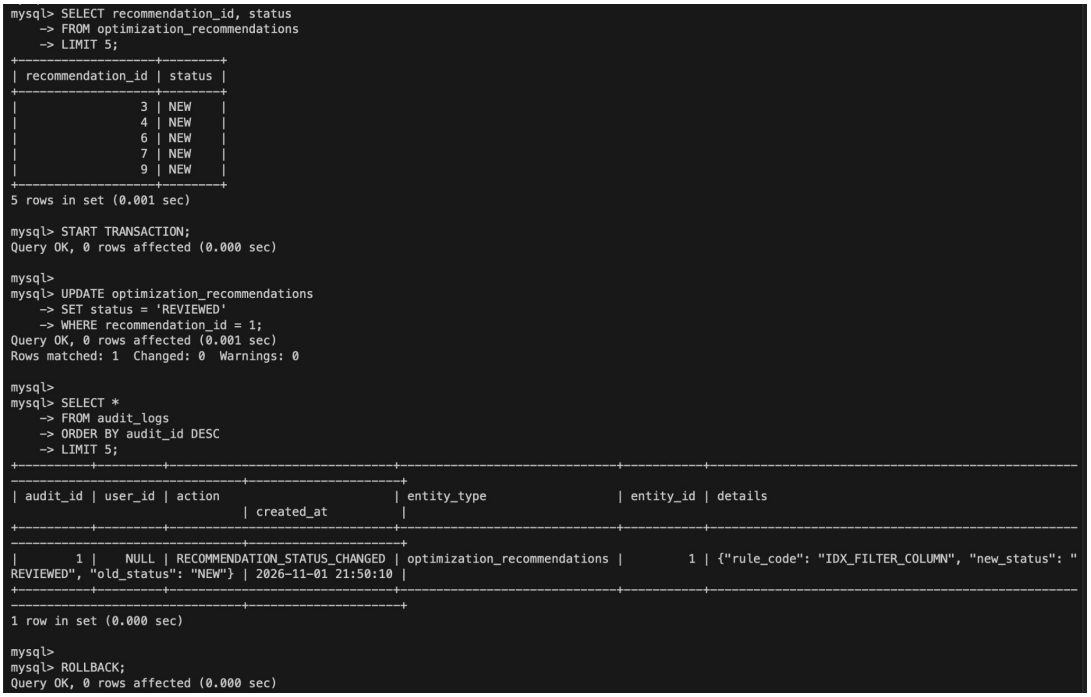
```
mysql> USE idop_demo_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
mysql> SELECT
    ->     c.full_name,
    ->     COUNT(o.order_id) AS total_orders,
    ->     SUM(o.total_amount) AS total_order_value
    -> FROM customers c
    -> LEFT JOIN orders o
    ->     ON c.customer_id = o.customer_id
    -> GROUP BY c.customer_id, c.full_name;
+--------------+--------------+-------------------+
| full_name    | total_orders | total_order_value |
+--------------+--------------+-------------------+
| Aarav Sharma |            3 |          11243.00 |
| Ananya Rao   |            2 |           6197.00 |
| Rohan Mehta  |            2 |           5098.00 |
| Isha Nair    |            1 |           1299.00 |
| Kabir Verma  |            1 |           3499.00 |
| Diya Iyer    |            2 |           5697.00 |
| Arjun Reddy  |            1 |           2199.00 |
| Meera Joshi  |            1 |           1599.00 |
| Vivaan Gupta |            1 |           2398.00 |
| Saanvi Menon |            1 |            499.00 |
+--------------+--------------+-------------------+
10 rows in set (0.001 sec)

mysql>
mysql>
mysql>
mysql>
mysql>
mysql> SELECT idop_demo_db.fn_order_item_total(1);
+--------------------------------------+
| idop_demo_db.fn_order_item_total(1) |
+--------------------------------------+
|                              4798.00 |
+--------------------------------------+
1 row in set (0.009 sec)

mysql>
mysql> CALL idop_demo_db.sp_reconcile_order_total(1);
Query OK, 0 rows affected (0.001 sec)
```

Figure 11. Trigger audit result — recommendation status-change audit record.



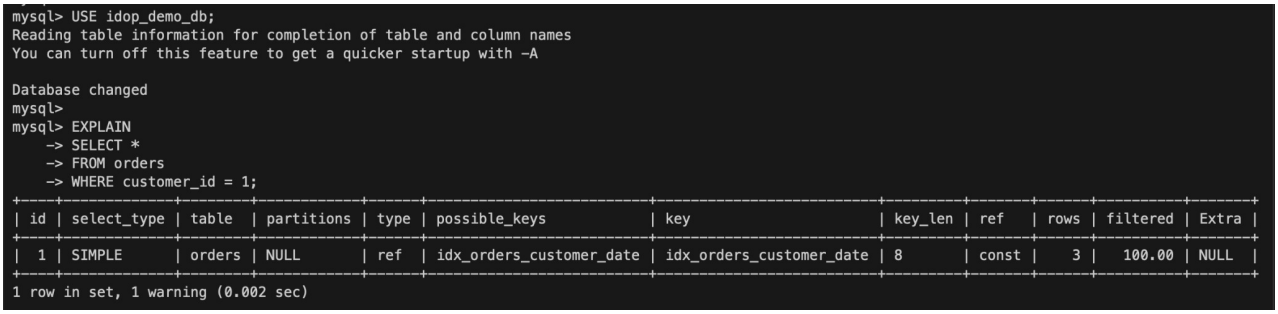
```
mysql> SELECT recommendation_id, status
    -> FROM optimization_recommendations
    -> LIMIT 5;
+-------------------+--------+
| recommendation_id | status |
+-------------------+--------+
|                 3 | NEW    |
|                 4 | NEW    |
|                 6 | NEW    |
|                 7 | NEW    |
|                 9 | NEW    |
+-------------------+--------+
5 rows in set (0.001 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

mysql>
mysql> UPDATE optimization_recommendations
    -> SET status = 'REVIEWED'
    -> WHERE recommendation_id = 1;
Query OK, 0 rows affected (0.001 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql>
mysql> SELECT *
    -> FROM audit_logs
    -> ORDER BY audit_id DESC
    -> LIMIT 5;
+----------+---------+-----------------------------+---------------------------+-----------+-----------------------------
```

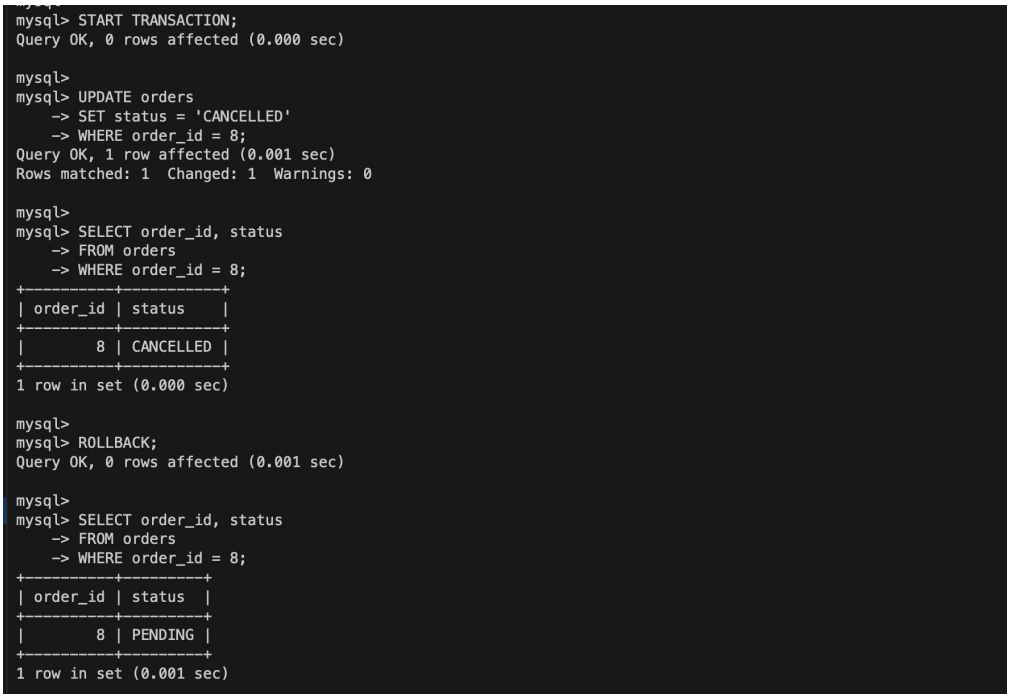
Figure 13. MySQL EXPLAIN — execution plan using idx_orders_customer_date.



```
mysql> USE idop_demo_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
mysql> EXPLAIN
    -> SELECT *
    -> FROM orders
    -> WHERE customer_id = 1;
+----+-------------+--------+------------+------+--------------------------+--------------------------+---------+-------+------+----------+-------+
| id | select_type | table  | partitions | type | possible_keys            | key                      | key_len | ref   | rows | filtered | Extra |
+----+-------------+--------+------------+------+--------------------------+--------------------------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | orders | NULL       | ref  | idx_orders_customer_date | idx_orders_customer_date | 8       | const |    3 |   100.00 | NULL  |
+----+-------------+--------+------------+------+--------------------------+--------------------------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.002 sec)
```

Figure 14. Transaction rollback — order 8 restored to PENDING after ROLLBACK.



```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

mysql>
mysql> UPDATE orders
    -> SET status = 'CANCELLED'
    -> WHERE order_id = 8;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SELECT order_id, status
    -> FROM orders
    -> WHERE order_id = 8;
+----------+-----------+
| order_id | status    |
+----------+-----------+
|        8 | CANCELLED |
+----------+-----------+
1 row in set (0.000 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.001 sec)

mysql>
mysql> SELECT order_id, status
    -> FROM orders
    -> WHERE order_id = 8;
+----------+---------+
| order_id | status  |
+----------+---------+
|        8 | PENDING |
+----------+---------+
1 row in set (0.001 sec)
```

Figure 15. Qdrant collection — healthy collection, four vectors, 384 dimensions, cosine distance.



11. Evaluation Rubrics – Completion Status

The status below is a team self-assessment mapped to the DBMS evaluation rubric. It describes implementation evidence, not marks awarded by the evaluator. Features not supported by the current evidence are marked for verification.

#	Criteria	Requirement	Status / evidence
1	Database Design	ER diagram and relational schema with keys, relationships, constraints	Implemented — Chen ERDs and relational summaries included.
2	Database Creation	SQL DDL, constraints, vector database creation, technology justification	Implemented — MySQL and Qdrant; stack justified.
3	SQL Operations and Queries	DML, nested/correlated/recursive SQL, joins	Partially evidenced — DML, scalar nested query and joins shown; verify correlated/recursive examples in submitted SQL.
4	Advanced Database Concepts	Procedures/functions, trigger, CTE, NoSQL/vector feature RUD	Implemented in part — routines, trigger, transactions, FULLTEXT, vector similarity shown; verify CTE and vector update/delete evidence.
5	Web Application and Database Integration	Next.js interface and backend/database integration	Implemented — Next.js, FastAPI, MySQL, Qdrant.
6	Authentication and Authorization	Authentication and minimum two roles with RBAC	Implemented — Viewer, Analyst, Admin; backend authorization.
7	Documentation, Git/GitHub, Demonstration, Testing, Originality	Documentation, repository, demo, testing, contribution	Implemented — README, source repository, screenshots, 10 Python tests, production build; explain team contributions in viva.

Repository and run instructions

GitHub repository: https://github.com/PiyushShiv26/DBMS-mini-project
Configure environment variables locally; do not commit database passwords, API secrets, or provisioning keys. Start MySQL and Qdrant first. Run the backend with `uvicorn backend.main:app --reload --port 8000`. In another terminal, run `cd frontend`, `npm install`, and `npm run dev`. Qdrant's local API is available on port 6333 with the documented Docker configuration.

Limitations and future enhancements

The SQL analyzer is rule-based and does not replace a complete SQL parser. The demonstration workload set is small and seeded. EXPLAIN output is not a measured performance benchmark. Future work includes broader workload ingestion, a full SQL parser, measured performance experiments, and automated end-to-end integration tests.

Team members

PES1UG24AM191 — Piyush Shiv
PES1UG24AM215 — Raghav G Uddihal